

HealthAI Coach

Tests de validation de la base de donnees

A executer dans phpMyAdmin — onglet SQL

Mode d'emploi : Ouvrez phpMyAdmin, selectionnez la base de donnees **healthai_coach**, cliquez sur l'onglet **SQL**, copiez la requete, collez-la et cliquez sur **Executer**. Verifiez que le resultat correspond au resultat attendu indique pour chaque test.

Test 01 Les 8 tables contiennent des donnees

Ce que fait cette requete :

Cette requete compte le nombre de lignes dans chacune des 8 tables de la base de donnees. Elle permet de verifier en une seule execution que le pipeline ETL a bien charge les donnees dans toutes les tables sans en oublier aucune.

Requete SQL :

```
SELECT 'users' AS table_name, COUNT(*) AS total FROM users
UNION SELECT 'biometrics', COUNT(*) FROM biometrics
UNION SELECT 'foods', COUNT(*) FROM foods
UNION SELECT 'exercises', COUNT(*) FROM exercises
UNION SELECT 'health_profiles', COUNT(*) FROM health_profiles
UNION SELECT 'user_goals', COUNT(*) FROM user_goals
UNION SELECT 'activity_logs', COUNT(*) FROM activity_logs
UNION SELECT 'nutrition_logs', COUNT(*) FROM nutrition_logs;
```

Resultat attendu :

✓ Chaque ligne doit afficher un nombre superieur a 0.

Si echec : Si une table affiche 0, le pipeline ETL n'a pas charge ses donnees.

Test 02 Tous les emails utilisateurs sont uniques

Ce que fait cette requete :

Cette requete compare le nombre total d'utilisateurs avec le nombre d'emails distincts. L'email etant la cle d'identification de chaque utilisateur, il ne doit pas y avoir de doublon. Un doublon signifierait que deux utilisateurs ont ete crees avec la meme adresse.

Requete SQL :

```
SELECT COUNT(*) AS total,  
COUNT(DISTINCT email) AS uniques  
FROM users;
```

Resultat attendu :

✓ **total et uniques doivent etre egaux (aucun doublon).**

Si echec : Si total > uniques, il existe des emails en double dans la table users.

Test 03 Toutes les biometries sont liees a un utilisateur existant

Ce que fait cette requete :

Cette requete cherche les enregistrements de la table biometrics dont le champ user_id ne correspond a aucun utilisateur dans la table users. Cela s'appelle une violation de cle etrangere — une donnee qui pointe vers un utilisateur inexistant.

Requete SQL :

```
SELECT COUNT(*) AS orphelins  
FROM biometrics  
WHERE user_id NOT IN (SELECT id FROM users);
```

Resultat attendu :

✓ **Le resultat doit etre 0.**

Si echec : Un resultat > 0 signifie qu'il existe des biometries sans utilisateur associe.

Test 04**Tous les activity_logs sont lies a un utilisateur existant****Ce que fait cette requete :**

Meme principe que le test 03, applique cette fois a la table activity_logs. Chaque seance d'activite doit obligatoirement etre rattachee a un utilisateur existant. Un enregistrement orphelin indique un probleme lors du chargement des donnees.

Requete SQL :

```
SELECT COUNT(*) AS orphelins
FROM activity_logs
WHERE user_id NOT IN (SELECT id FROM users);
```

Resultat attendu :

✓ **Le resultat doit etre 0.**

Si echec : Un resultat > 0 indique une violation de cle etrangere dans activity_logs.

Test 05**Les valeurs du champ genre sont valides****Ce que fait cette requete :**

Le champ genre est un ENUM : il n'accepte que les valeurs 'M', 'F' ou 'autre'. Cette requete liste toutes les valeurs distinctes presentes dans la table users pour verifier qu'aucune valeur inattendue n'a ete inseree lors du pipeline ETL.

Requete SQL :

```
SELECT DISTINCT genre
FROM users;
```

Resultat attendu :

✓ **Seules les valeurs M, F ou autre doivent apparaitre.**

Si echec : Toute autre valeur indique un probleme de transformation dans le pipeline ETL.

Test 06**Toutes les calories des aliments sont positives ou nulles****Ce que fait cette requete :**

Les calories d'un aliment ne peuvent pas etre negatives. Cette requete detecte les lignes de la table foods dont la valeur calories_kcal est inferieure a zero, ce qui traduirait une erreur lors du nettoyage des donnees CSV.

Requete SQL :

```
SELECT COUNT(*) AS calories_negatives
FROM foods
WHERE calories_kcal < 0;
```

Resultat attendu :

✓ **Le resultat doit etre 0.**

Si echec : Des calories negatives indiquent une erreur de nettoyage dans le fichier CSV.

Test 07**Tous les BMI sont dans une plage realiste****Ce que fait cette requete :**

L'Indice de Masse Corporelle (BMI) d'un etre humain se situe normalement entre 10 et 80. Cette requete detecte les valeurs aberrantes dans la table biometrics qui seraient en dehors de cette plage physiologiquement plausible.

Requete SQL :

```
SELECT COUNT(*) AS aberrants
FROM biometrics
WHERE bmi IS NOT NULL
AND (bmi < 10 OR bmi > 80);
```

Resultat attendu :

✓ **Le resultat doit etre 0.**

Si echec : Un BMI < 10 ou > 80 est physiologiquement improbable et indique une anomalie.

Test 08**Chaque utilisateur a au moins un objectif****Ce que fait cette requete :**

Dans le modele de donnees, chaque utilisateur doit avoir au moins un objectif (user_goals). Cette requete utilise une jointure LEFT JOIN pour identifier les utilisateurs pour lesquels aucun objectif n'a ete insere dans la table user_goals.

Requete SQL :

```
SELECT COUNT(*) AS sans_objectif
FROM users u
LEFT JOIN user_goals g ON u.id = g.user_id
WHERE g.id IS NULL;
```

Resultat attendu :

✓ **Le resultat doit etre 0.**

Si echec : Un resultat > 0 signifie que certains utilisateurs n'ont pas d'objectif associe.

Test 09**Les types de repas dans nutrition_logs sont valides****Ce que fait cette requete :**

Le champ repas dans nutrition_logs est un ENUM qui accepte uniquement : petit_dejeuner, dejeuner, diner, collation. Cette requete liste toutes les valeurs distinctes presentes pour s'assurer qu'aucune valeur invalide n'a ete inseree.

Requete SQL :

```
SELECT DISTINCT repas
FROM nutrition_logs;
```

Resultat attendu :

✓ **Seules les valeurs petit_dejeuner, dejeuner, diner, collation doivent apparaitre.**

Si echec : Toute autre valeur indique une erreur de mapping dans le pipeline ETL.

Test 10**Aucun aliment sans nom****Ce que fait cette requete :**

Le nom est le champ principal d'identification d'un aliment dans la table foods. Cette requete verifie qu'aucun aliment n'a ete insere avec un nom vide ou NULL, ce qui rendrait les donnees inexploitablees pour les recommandations nutritionnelles.

Requete SQL :

```
SELECT COUNT(*) AS sans_nom  
FROM foods  
WHERE nom IS NULL OR nom = '';
```

Resultat attendu :

✓ **Le resultat doit etre 0.**

Si echec : Un aliment sans nom est une donnee incomplete qui doit etre corrige.